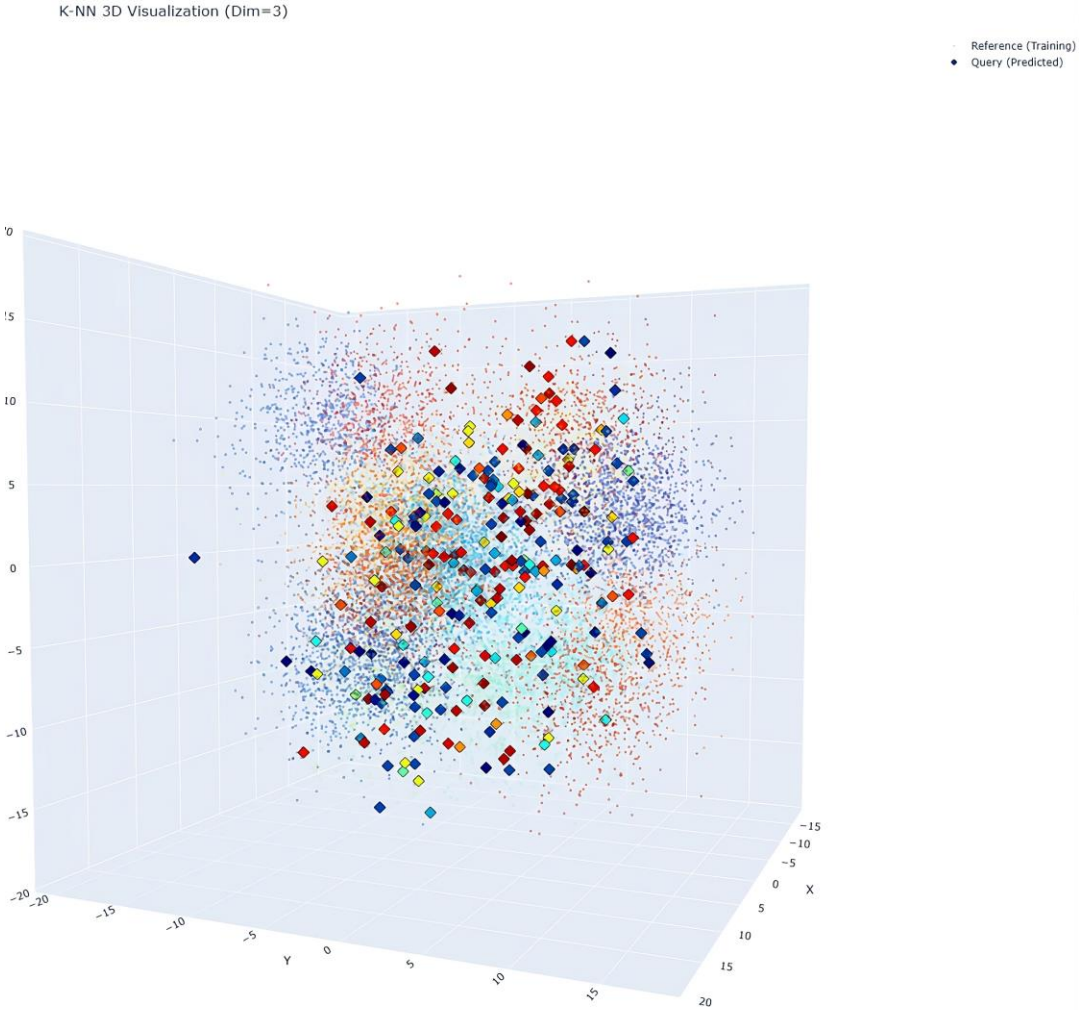
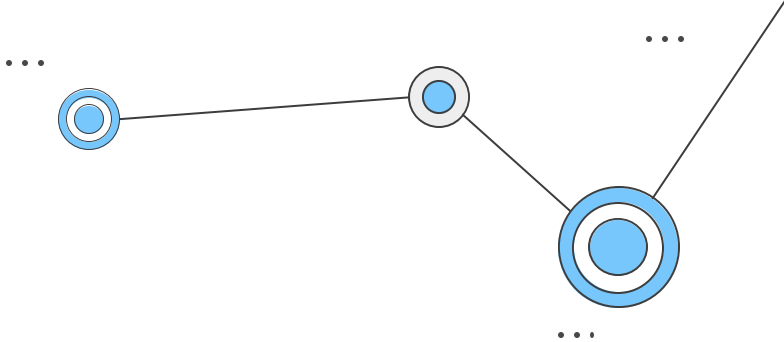


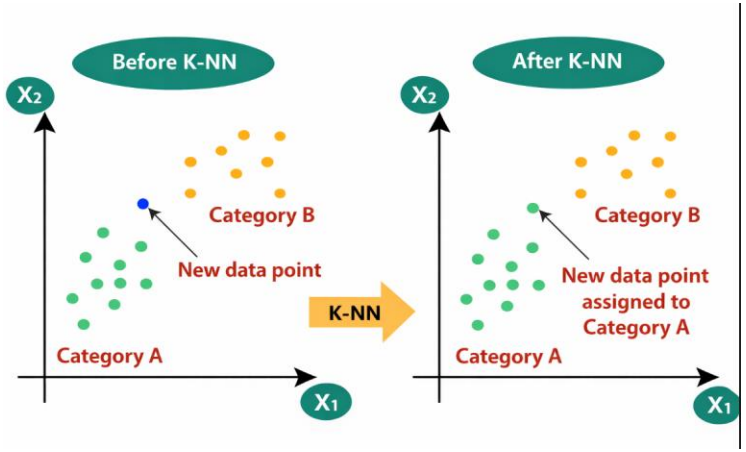


Algoritmo k-NN

Simone Pipola,
Ulisse Steccanella

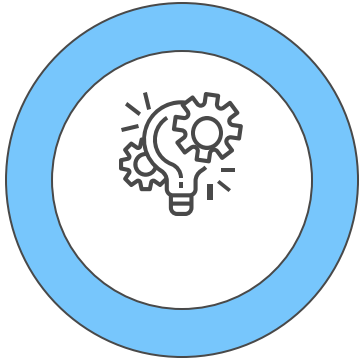
Il Problema

Problema:
Dato un dataset di punti già classificati con posizione e colore. L'algoritmo k-NN ha lo scopo di assegnare una classe colore a nuovi punti in input basandosi sulle caratteristiche dei k punti più vicini.



Tre step fondamentali:

1. Calcolo delle distanze (Distanza Euclidea)
2. Ordinamento
3. Votazione

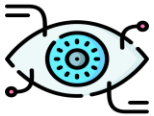


Problema vs. Soluzione

Problema: Complessità O (M * N) e ampi tempi di calcolo sequenziali per dataset grandi.
Soluzione: Sfruttare l'architettura SIMT delle GPU per un parallelismo massivo delle operazioni indipendenti.

...

Applicazioni pratiche

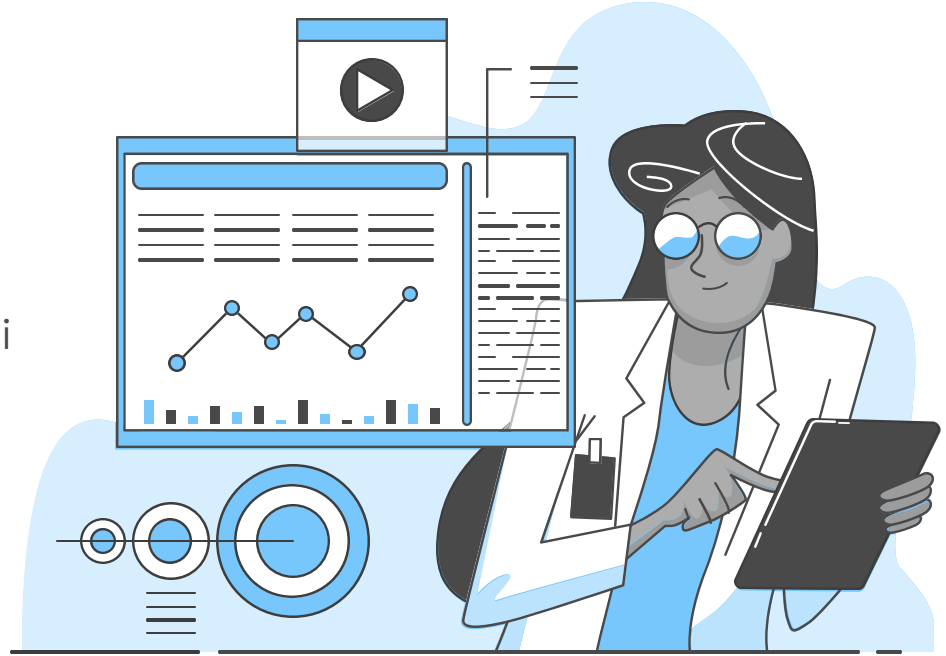


Computer Vision
Riconoscimento di pattern

Bioinformatica
Analisi sequenze genetiche

Venditori di Servizi
Suggerimento prodotti simili

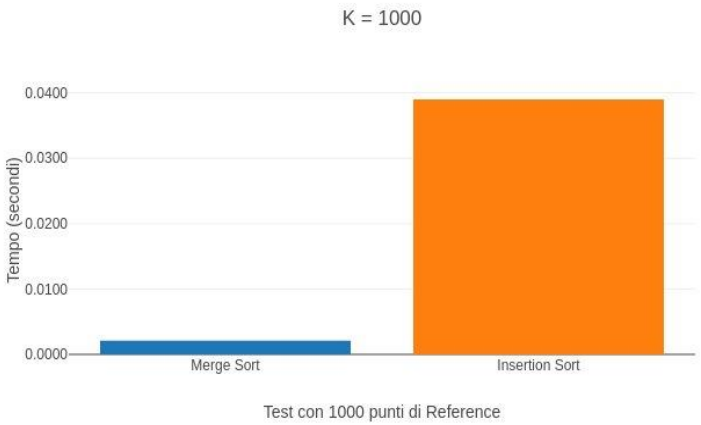
Data Mining
Analisi dataset complessi

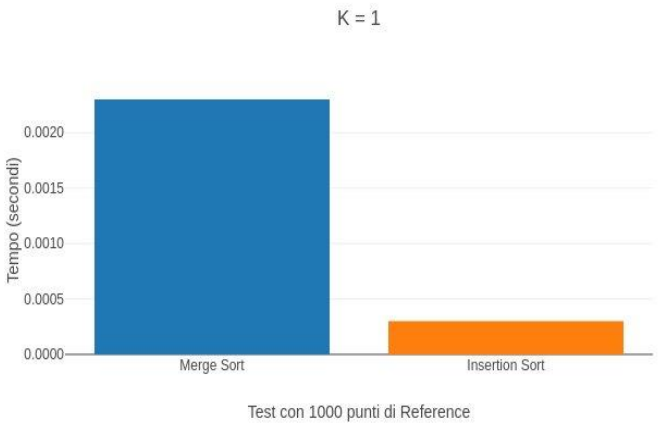


Codice sequenziale: risultati sperimentali

- Se K è molto grande (circa uguale al numero di punti di Reference), occorre riordinare la maggior parte delle distanze calcolate tramite Merge Sort

- Se K è molto piccolo (<< numero di punti di Reference), si può tenere traccia delle K distanze più piccole calcolate, senza salvarle tutte in memoria, usando Insertion Sort





CUDA: prima versione "naive"

Caratteristiche:

- 1 blocco da 256 thread
- Un kernel in cui ogni thread computa le distanze tra un punto di query e tutti i punti reference
- Una funzione della libreria CUB per fare merge sort delle distanze calcolate, per ogni punto di query
- Un kernel per estrarre le K distanze più piccole e decisione del colore

Tempo di esecuzione: 0.610866s (Ref.=1000000, K=1)

Problemi:

- *Small Grid*: La griglia è configurata per eseguire 1 blocco, mentre la GPU avrebbe a disposizione 40 SM
- *Achieved Occupancy* (12.5 %): eccessivo overhead nello scheduling dei warp oppure workload non bilanciato tra i warp.
- *Long Scoreboard Stalls* (11 cicli di stallo per warp): le modalità di accesso alla memoria non sono ottimali, basso HIT rate in cache oppure accessi non coalescenti

Fase di Ottimizzazione: Griglia 2D

Caratteristiche:

- blocchi 2D 16x16
- griglia 2D in modo tale che i blocchi coprano un'area di N_REF * N_QUERY elementi
- Un kernel in cui ogni thread computa UNA distanza tra un punto di query e un punto di reference

Tempo di esecuzione: 0.095657s (Ref.=1000000, K=1)

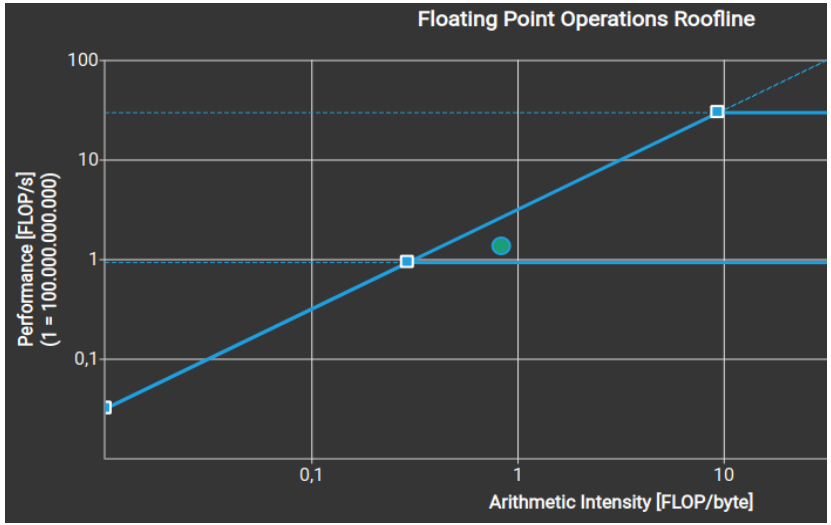
```
dim3 blockDim2D(16, 16);
dim3 gridDim2D(
    (N_REF   + blockDim2D.x - 1) / blockDim2D.x,
    (N_QUERY + blockDim2D.y - 1) / blockDim2D.y
);
```

```
// Calcolo distanze euclidee (senza sqrt)
for(int d = 0; d < dim; d++) {
    float diff = query_points[q_idx * dim + d] - ref_points[r_idx * dim + d];
    distance_sq += diff * diff;
}
```

Fase di Ottimizzazione: Griglia 2D

Problemi:

- *Long Scoreboard Stalls* (11.7 cicli di stallo per warp): le modalità di accesso alla memoria non sono ottimali, basso HIT rate in cache oppure accessi non coalescenti
- *L1TEX Global Load Access Pattern* (11.7B utilizzati / 32 B trasmessi): Accessi non ottimali alla cache, possibili accessi stride tra thread
- *Uncoalesced Global Accesses* (12% dei settori portati da L2 a L1 sono inutili): possibili accessi sparsi alla cache, troppi settori coinvolti rispetto a quelli richiesti



Theoretical Occupancy [%]	100
Theoretical Active Warps per SM [warp]	32
Achieved Occupancy [%]	85,11
Achieved Active Warps Per SM [warp]	27,24

Fase di Ottimizzazione: Tiling con shared memory

Caratteristiche:

- Ogni Thread Block si occupa di una sottomatrice del problema (tile). I suoi thread caricano i vettori *Query* e *Reference* necessari in SMEM una sola volta per blocco, permettendo un riutilizzo dei dati pari alla dimensione del tile.
- I thread collaborano per effettuare il pre-fetching dei dati dalla Global Memory alla SMEM. Questo trasforma accessi ridondanti e sparsi alla DRAM in letture burst verso la memoria locale
- Una volta in SMEM, l'accesso ai dati per il calcolo della distanza euclidea avviene a latenza quasi nulla

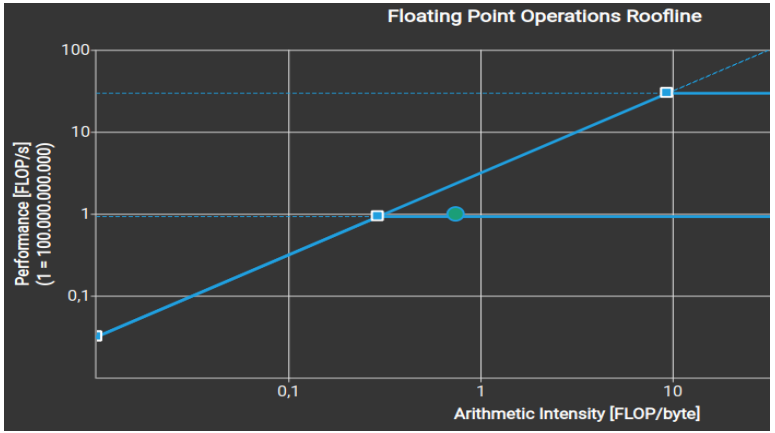
Tempi di esecuzione: 0.097352s (Ref.=1000000, K=1)

```
// Caricamento in Shared Memory (se entro i limiti)
if (row < n_query && tx == 0) {
    for(int d=0; d < dim; d++)
        s_query[ty * dim + d] = query_points[row * dim + d];
}

if (col < n_ref && ty == 0) {
    for(int d=0; d < dim; d++)
        s_ref[tx * dim + d] = ref_points[col * dim + d];
    s_ref_labels[tx] = ref_labels[col];
}

__syncthreads();
```

Fase di Ottimizzazione: Tiling con shared memory

Problematiche:

- *L1TEX Global Load Access Pattern* (9.8B utilizzati / 32 B trasmessi): Accessi non ottimali alla cache, possibili accessi stride tra thread
- *Barrier Stalls* (5.8 cicli di stallo per warp): stalli per barriere di sincronizzazione
- *Thread divergence*: 21.7 thread attivi per ciclo di clock



Theoretical Occupancy [%]	100
Theoretical Active Warps per SM [warp]	32
Achieved Occupancy [%]	90,40
Achieved Active Warps Per SM [warp]	28,93

Fase di Ottimizzazione: Accessi Coalescenti

Caratteristiche:

- Coordinate organizzate per dimensione [x1,x2,..y1,y2,..z1,z2..] invece che per punto [x1,y1,z1,..,x2,y2,z2,..] così che tutti i thread adiacenti (warp) accedano ora a indirizzi di memoria contigui (es. tutti leggono la coordinata X di punti consecutivi).
- Gli accessi sono perciò coalescenti

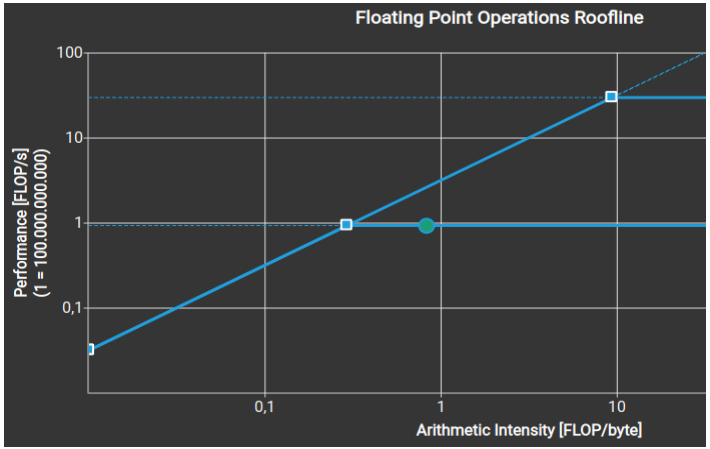
Tempi di esecuzione: 0.098254s (Ref.=1000000, K=1)

```
    // --- ACCESSO MEMORIA OTTIMIZZATO (SoA) ---
    // ref_points è organizzato come: [Tutte le Dim0 | Tutte le Dim1 | Tutte le Dim2...]

    for(int d = 0; d < dim; d++) {
        // Calcolo indici per SoA
        // Per Ref: saltiamo n_ref posizioni per cambiare dimensione
        float ref_val = ref_points[d * n_ref + r_idx];

        // Per Query: possiamo lasciare AoS (tanto è letta in broadcast/cache)
        // oppure convertire anche lei in SoA. Se AoS:
        float query_val = query_points[d * n_query + q_idx];

        float diff = query_val - ref_val;
        distance_sq += diff * diff;
    }

    size_t base_idx = q_idx * n_ref + r_idx;
    out_dists[base_idx]  = distance_sq;
    out_labels[base_idx] = ref_labels[r_idx];
}
```

Fase di Ottimizzazione: Accessi Coalescenti

Problematiche:

- *Long Scoreboard Stalls* (26.1 cicli di stallo per warp): le modalità di accesso alla memoria non sono ottimali, basso HIT rate in cache oppure accessi non coalescenti
- *Achieved Occupancy* (89.8%): alcuni warp oppure alcuni blocchi impiegano più tempo di altri ad eseguire
- *L1TEX Global Load Access Pattern* (25.5B utlizzati / 32 B trasmessi): Accessi non ottimali alla cache, possibili accessi stride tra thread



Theoretical Occupancy [%]	100
Theoretical Active Warps per SM [warp]	32
Achieved Occupancy [%]	89,82
Achieved Active Warps Per SM [warp]	28,74

Fase di Ottimizzazione: Insertion Sort

Caratteristiche:

- Local Top-K: Ogni thread identifica i propri k vicini locali salvandoli nei registri tramite Insertion Sort.
- Struttura ad Albero (Tree): I thread all'interno di un blocco collaborano fondendo le proprie liste a coppie in modo logaritmico.
- Dopo log_2(threads) passi, il thread 0 scrive i k vicini definitivi nel buffer di output.

Tempi di esecuzione: 0.003402s (Ref.=1000000, K=1)

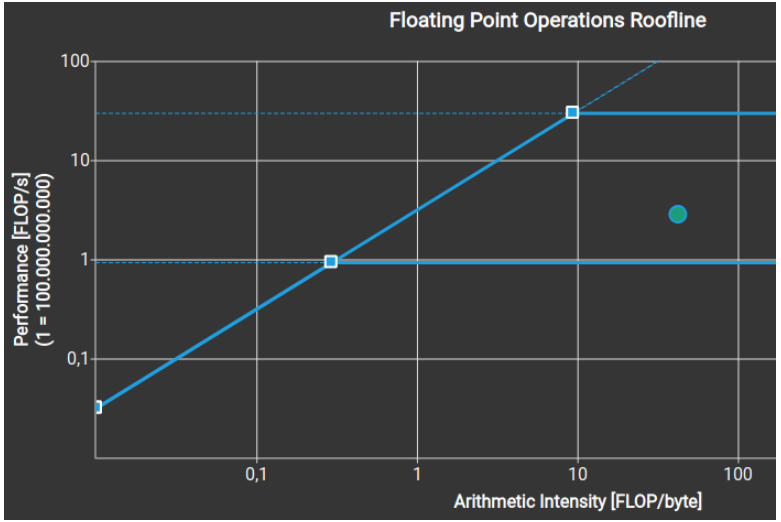
```
dim3 gridDim(N_QUERY, 1, 1);
dim3 blockDim(256, 1, 1);
size_t shared_mem_size = (DIM * sizeof(float)) +
    (blockDim.x * K * sizeof(float)) +
    (blockDim.x * K * sizeof(int));
```

```
// Buffer per la query e le distanze calcolate
extern __shared__ float s_query[];

// Caricamento cooperativo query
for (int d = tid; d < dim; d += blockDim.x) {
    s_query[d] = query_points[q_idx * dim + d];
}
__syncthreads();
```

Fase di Ottimizzazione: Insertion Sort

Problematiche:

- *Long Scoreboard Stalls* (9.9 cicli in attesa): Attesa latenza memoria globale.
- *L1TEX Local Load Access* (1 bytes di 32): Possibili accessi in lettura non coalescenti.
- *L1TEX Local Store Access* (2 bytes di 32): Possibili scritture in memoria non coalescenti.



Theoretical Occupancy [%]	100
Theoretical Active Warps per SM [warp]	32
Achieved Occupancy [%]	62,50
Achieved Active Warps Per SM [warp]	20,00

Confronto dati sperimentali

	K=1	K=23	K=1000
Ottimizzato	0.95657s	0.104509s	0.104685s
Tiling e SMEM	0.097352s	0.106997s	0.104846s
Accessi coalescenti	0.098254s	0.107324s	0.105828s
Insertion Sort	0.003402s	0.056672s	X

Insertion Sort, problema per K >> 1:

SMEM_occupata = query_dim + buffer_distanze_dim + buffer_labels_dim =
= 3*4 + 256*K*4 + 256*K*4

K=23: 3*4 + 256*23*4 + 256*23*4 = **47116 B** ≃ 48000 B (SMEM per blocco)

Verso una strategia Dual Path

- **Versioni con Merge Sort: Ottimizzato_1, Tiling con SMEM e accessi coalescenti:** ogni thread calcola UNA distanza tra un punto di query e un punto reference.
- **Versione con Insertion Sort:**
  ogni blocco calcola le distanze per una query. Uso di SMEM per punti query e reference.

LOW K

Griglia 2D, Blocchi 2D

Per ogni blocco, ogni thread mantiene una lista di K distanze, che vengono unite alla fine.

HIGH K

Griglia 1D, Blocchi 1D

Tutte le distanze calcolate vengono ordinate con Merge Sort della libreria CUB

Grazie

Domande?

CREDITS: This presentation template was created by Slidesgo, including icons by Flaticon, infographics & images by Freepik and illustrations by Stories